Lecture 20: Simple Mail Transfer Protocol, SMTP (*RFC 821*) -II

Objectives:

- Learn C# classes for handling e-mails (MailMessage & MailAttachment classes)
- Learn how to write a simple SMTP DLL and a simple mailing application

1. C# classes for handling e-mails

C# provides three classes in the System.Web.Mail namespace for handling mails, namely: SmtpMail, MailMessage and MailAttachment.

However, SmtpMail class is specifically designed for using the SMTP service provided in IIS, making its use limited.

We shall use the other two classes to develop our own SmtpClient class which will be more general.

1.1. MailMessage class

This class is used to represent a mail as an object. It has a number of properties which can be used to construct the mail. Some of these are:

String From	Gets or sets the from address
String To	Gets or sets the semicolon-delimited list of to addresses
String Subject	Gets or sets the string for the subject
String Cc	Gets or sets the semicolon-delimited list of cc addresses
String Bcc	Gets or sets the semicolon-delimited list of bcc addresses
String Body	Gets or sets the body of the message
IList Attachments	Specifies a list of file attachments to add to the message

To construct a mail message, just create an instance of MailMessage using its default constructor and then assign values to its properties.

1.2. MailAttachement class

To simplify e-mail attachment, the .NET library includes the MailAttachment class.

This class specifies the content and format of an attachment to be included in the mail.

It has two constructors as follows:

MailAttachment(string filename)
MailAttachment(string filename, MailEncoding ecncoding)

The first constructor uses UUEncoding by default, while the second allows the desired encoding to be specified. The options are: MailEncoding.Base64 for MIME encoding and MailEncoding.UUEncode for uuencoding.

We shall be using the Base64 encoding.

2. Simple SmtpClient DLL

The idea of SmtpClient class is to provide a high-level class that a user can use to send his message.

The user should only need to supply the login-details for his SMTP server and the message he wishes to send.

The class should establish a connection with the SMTP server and handle all the necessary SMTP exchanges with the server to send the message.

The following is a simple implementation of SmtpClient class. It requires the client to provide server login details when creating an instance of the class (through the contructor).

The class provides a public method :

```
public bool Send(MailMessage message)
```

which the user can use to send his message.

Note that the SmtpClient class assumes that the MailMessage may contain MailAttachments. However, to understand how Attachements are handled, we must study MIME protocol. MIME is discussed in a subsequent lecture.

```
using System;
using System.Text;
using System.IO;
using System.Net.Sockets;
using System.Net;
using System.Web.Mail;
namespace SMTP
{
     public class SmtpClient
     {
          private string smtpServer;
          private string username;
          private string password;

          public SmtpClient(string server)
          {
               smtpServer = server;
          }

          public SmtpClient(string server, string  user, string pass)
          {
               smtpServer = server;
```

```
                username = user;
                password = pass;
          }

          private enum SMTPResponse: int
          {
                CONNECT_SUCCESS = 220,
                GENERIC_SUCCESS = 250,
                DATA_SUCCESS = 354,
                QUIT_SUCCESS = 221,
                EXCHANGE_SUCCESS = 334,
                LOGIN_SUCCESS = 235
          }

          private void SendData(Socket s, string msg)
          {
                byte[] data = Encoding.ASCII.GetBytes(msg);
                s.Send(data , 0, data.Length, SocketFlags.None);
          }

          private bool CheckResponse(Socket s, SMTPResponse
expectedResponse )
          {
                string sResponse;
                int response;
                byte[] bytes = new byte[1024];
                int size = s.Receive(bytes, 0, bytes.Length,
SocketFlags.None);
                sResponse = Encoding.ASCII.GetString(bytes,0, size);
                response = int.Parse(sResponse.Substring(0,3));
                if(response != (int)expectedResponse)
                     return false;
                return true;
          }

          public bool Send(MailMessage message)
          {
                IPHostEntry IPhst = Dns.Resolve(smtpServer);
                IPEndPoint endPt = new IPEndPoint(IPhst.AddressList[0],
25);
                Socket s= new Socket(AddressFamily.InterNetwork,
SocketType.Stream,ProtocolType.Tcp);
                s.Connect(endPt);

                if(!CheckResponse(s, SMTPResponse.CONNECT_SUCCESS))
                {
                     s.Close();
                     return false;
                }

                SendData(s, "HELO "+Dns.GetHostName()+"\r\n");
                if(!CheckResponse(s, SMTPResponse.GENERIC_SUCCESS))
                {
```

```
                    s.Close();
                    return false;
               }

              //if authentication is required
               if (username != null)
               {
                    SendData(s, "AUTH LOGIN\r\n");
                    CheckResponse(s, SMTPResponse.EXCHANGE_SUCCESS);
                    SendData(s,
Convert.ToBase64String(Encoding.ASCII.GetBytes(username))+"\r\n");
                    CheckResponse(s, SMTPResponse.EXCHANGE_SUCCESS);
                    SendData(s,
Convert.ToBase64String(Encoding.ASCII.GetBytes(password))+"\r\n");
                    if(!CheckResponse(s, SMTPResponse.LOGIN_SUCCESS))
                    {
                         s.Close();
                         return false;
                    }
               }

               SendData(s, "MAIL From: "+message.From+"\r\n");
               if(!CheckResponse(s, SMTPResponse.GENERIC_SUCCESS))
               {
                    s.Close();
                    return false;
               }

               string[] Tos=  message.To.Split(new char[] {';'});
               foreach (string To in Tos)
               {
                    SendData(s, "RCPT TO: " +To+ "\r\n");
                    if(!CheckResponse(s, SMTPResponse.GENERIC_SUCCESS))
                    {
                         s.Close();
                         return false;
                    }
               }

               if(message.Cc!=null)
               {
                    Tos= message.Cc.Split(new char[] {';'});
                    foreach (string To in Tos)
                    {
                         SendData(s, "RCPT TO: " +To+ "\r\n");
                         if(!CheckResponse(s,
SMTPResponse.GENERIC_SUCCESS))
                         {
                              s.Close();
                              return false;
                         }
                    }
               }
```

```
                StringBuilder data=new StringBuilder();
                data.Append("From: " + message.From + "\r\n");
                Tos= message.To.Split(new char[] {';'});
                data.Append("To: ");
                for( int i=0; i< Tos.Length; i++)
                {
                     data.Append( i > 0 ? "," : "" );
                     data.Append(Tos[i]);
                }
                data.Append("\r\n");
                if(message.Cc!=null)
                {
                     Tos= message.Cc.Split(new char[] {';'});
                     data.Append("Cc: ");
                     for( int i=0; i< Tos.Length; i++)
                     {
                          data.Append( i > 0 ? "," : "" );
                          data.Append(Tos[i]);
                     }
                     data.Append("\r\n");
                }
                data.Append( "Date: " );
                data.Append(DateTime.Now.ToString("ddd, d M y H:m:s z" ));
                data.Append("\r\n");
                data.Append("Subject: " + message.Subject+ "\r\n");
                data.Append( "X-Mailer: SMTPDirect v1\r\n" );
                string MsgBody = message.Body;
                if(!MsgBody.EndsWith("\r\n"))
                     MsgBody+="\r\n";

                //***************begin process attachments*************

                if(message.Attachments.Count>0)
                {
                     data.Append( "MIME-Version: 1.0\r\n" );
                     data.Append( "Content-Type: multipart/mixed;
boundary=unique-boundary-1\r\n" );
                     data.Append("\r\n");
                     data.Append( "This is a multi-part message in MIME
format.\r\n" );
                     StringBuilder sb = new StringBuilder();
                     sb.Append("--unique-boundary-1\r\n");
                     sb.Append("Content-Type: text/plain\r\n");
                     sb.Append("Content-Transfer-Encoding: 7Bit\r\n");
                     sb.Append("\r\n");
                     sb.Append(MsgBody + "\r\n");
                     sb.Append("\r\n");

                     foreach(object obj in message.Attachments)
                     {
                          MailAttachment a = obj as MailAttachment;
                          byte[] binaryData;
```

```
                              if(a!=null)
                              {
                                   FileInfo f = new FileInfo(a.Filename);
                                   sb.Append("--unique-boundary-1\r\n");
                                   sb.Append("Content-Type: application/octet-
stream; file=" + f.Name + "\r\n");
                                   sb.Append("Content-Transfer-Encoding:
base64\r\n");
                                   sb.Append("Content-Disposition: attachment;
filename=" + f.Name + "\r\n");
                                   sb.Append("\r\n");
                                   FileStream fs = f.OpenRead();
                                   binaryData = new Byte[fs.Length];
                                   long bytesRead = fs.Read(binaryData, 0,
(int)fs.Length);
                                   fs.Close();
                                   string base64String =
System.Convert.ToBase64String(binaryData, 0,binaryData.Length);

                                   for(int i=0; i< base64String.Length ; )
                                   {
                                        int nextchunk=100;
                                        if(base64String.Length - (i + nextchunk
) <0)
                                             nextchunk = base64String.Length -
i;
                                        sb.Append(base64String.Substring(i,
nextchunk));
                                        sb.Append("\r\n");
                                        i+=nextchunk;
                                   }
                                   sb.Append("\r\n");
                              }
                         }
                         sb.Append("--unique-boundary-1--\r\n");
                         MsgBody=sb.ToString();
                    }
                    //*************end process attachments***********

                    SendData(s, ("DATA\r\n"));
                    if(!CheckResponse(s, SMTPResponse.DATA_SUCCESS))
                    {
                         s.Close();
                         return false;
                    }
                    data.Append( "\r\n" );
                    data.Append( MsgBody );
                    data.Append( ".\r\n" );
                    data.Append( "\r\n" );
                    data.Append( "\r\n" );
                    SendData(s, data.ToString());
                    if(!CheckResponse(s, SMTPResponse.GENERIC_SUCCESS ))
                    {
```

```
                    s.Close();
                    return false;
                }

                SendData(s, "QUIT\r\n");
                CheckResponse(s, SMTPResponse.QUIT_SUCCESS );
                s.Close();
                return true;
            }
        }
}
```

Note: You may not understand the code between the commented lines, "//**********" until we study MIME in a subsequent lecture.

3. Simple Mail Application:

The following example makes use of the **SmtpClient** class to implement a simple mailing application.

```
using System;
using System.Windows.Forms;
using System.Web.Mail;
using SMTP;

namespace MyFormProject
{
    class SimpleMailer : System.Windows.Forms.Form
    {
        private System.Windows.Forms.TextBox passBox;
        private System.Windows.Forms.Button btSend;
        private System.Windows.Forms.Label label6;
        private System.Windows.Forms.Label label5;
        private System.Windows.Forms.Label label4;
        private System.Windows.Forms.Label label3;
        private System.Windows.Forms.Label label2;
        private System.Windows.Forms.Label label;
        private System.Windows.Forms.TextBox toBox;
        private System.Windows.Forms.TextBox userBox;
        private System.Windows.Forms.TextBox subjectBox;
        private System.Windows.Forms.TextBox serverBox;
        private System.Windows.Forms.TextBox message;
        private System.Windows.Forms.TextBox fromBox;

        public SimpleMailer() {
            InitializeComponent();
        }

        void InitializeComponent() {
```

```
                //deleted
            }
            [STAThread]
            public static void Main(string[] args)  {
                Application.Run(new SimpleMailer());
            }
        void btSendClick(object sender, System.EventArgs e)
            {

                MailMessage mail = new MailMessage();
                mail.From = fromBox.Text;
                mail.To = toBox.Text;
                mail.Subject = subjectBox.Text;
                mail.Body = message.Text;

                SmtpClient client = new SmtpClient(serverBox.Text,
userBox.Text, passBox.Text);
                if (client.Send(mail))
                    MessageBox.Show("Mail Sent Successfully");
                else
                    MessageBox.Show("Error Sending Message");
            }


        }
}
```